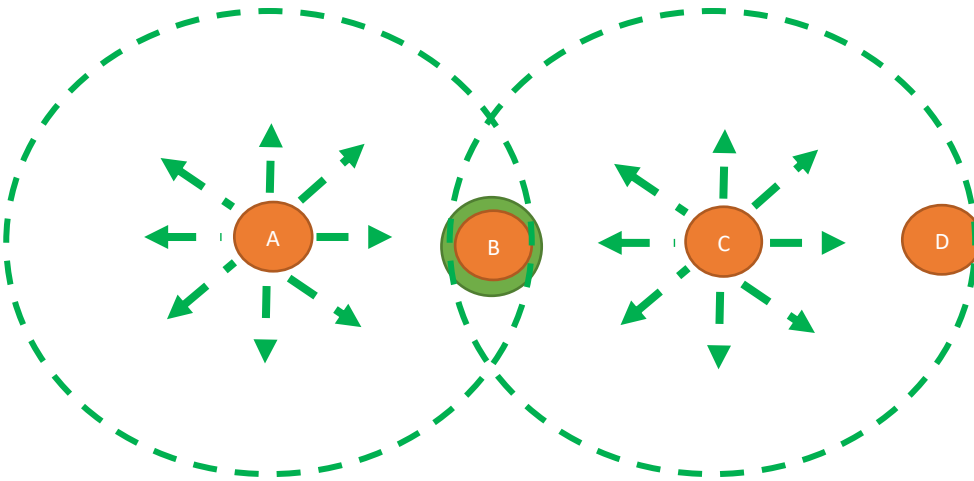
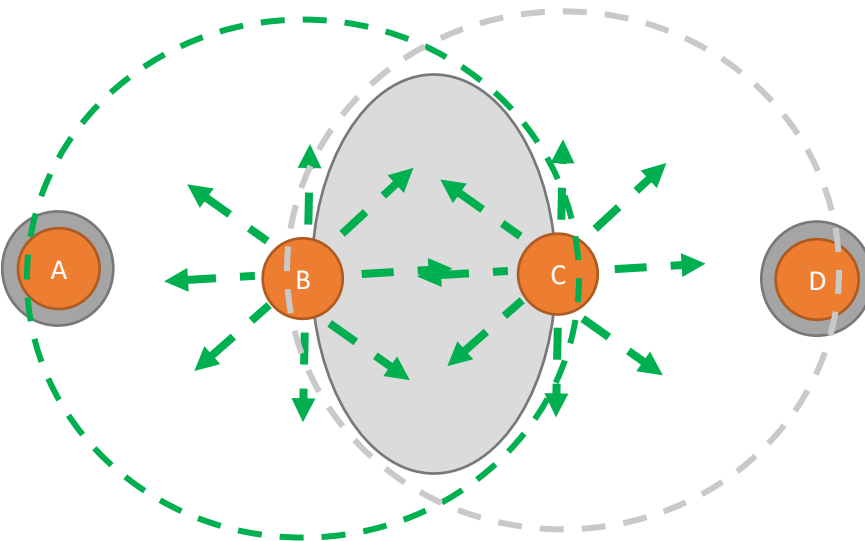


At the oral exam I will start the exam asking you to comment one of these schemas:

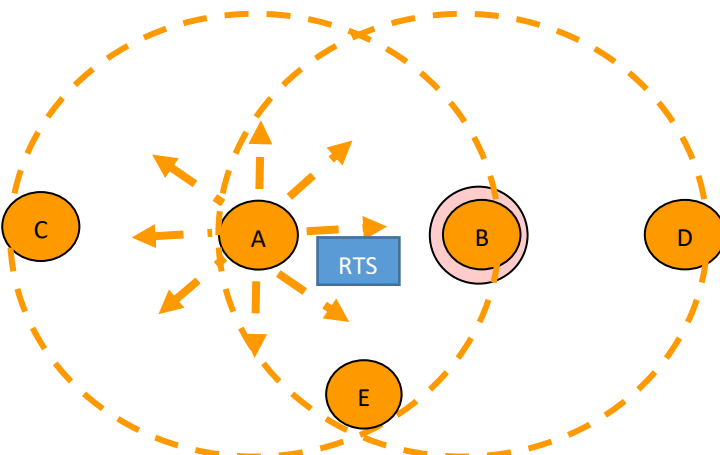
hidden terminal



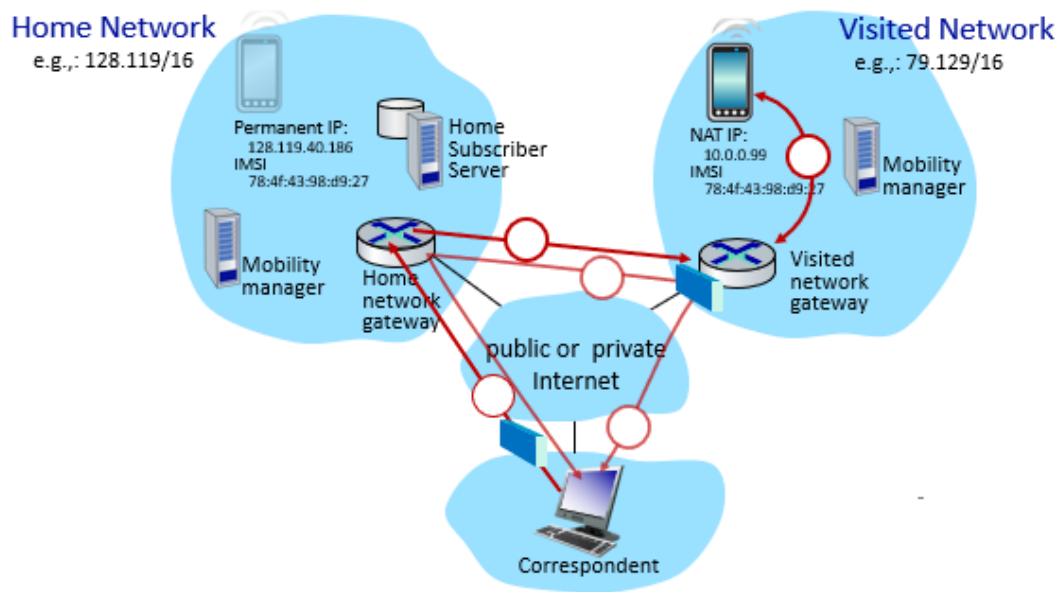
exposed terminal



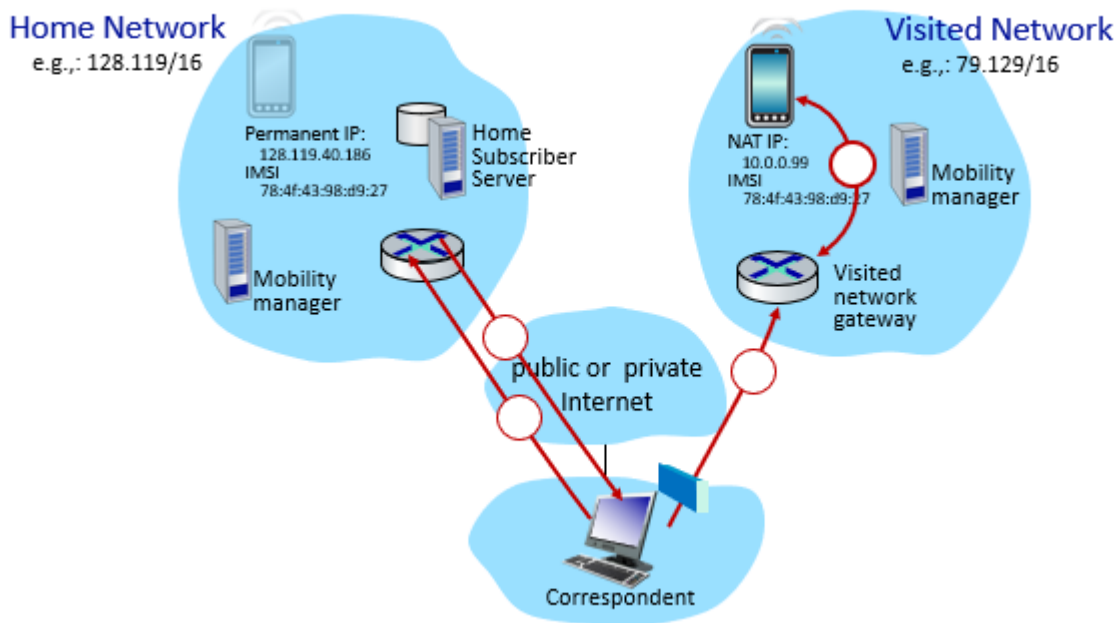
RTS/CTS



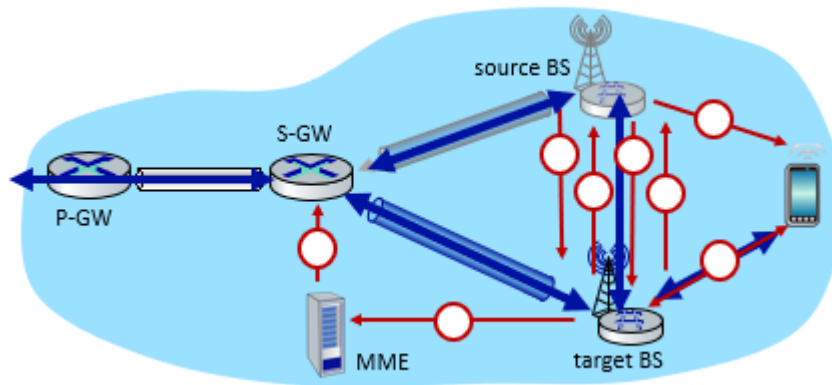
Mobile networks – I



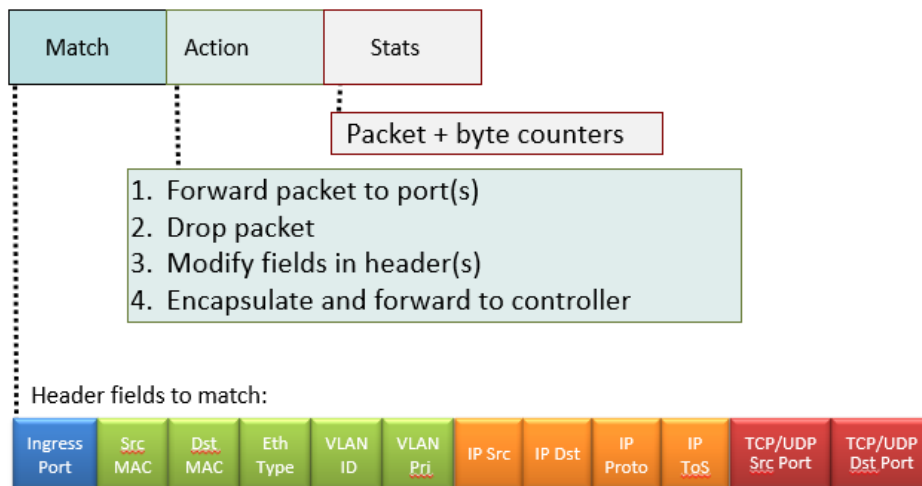
Mobile networks - II



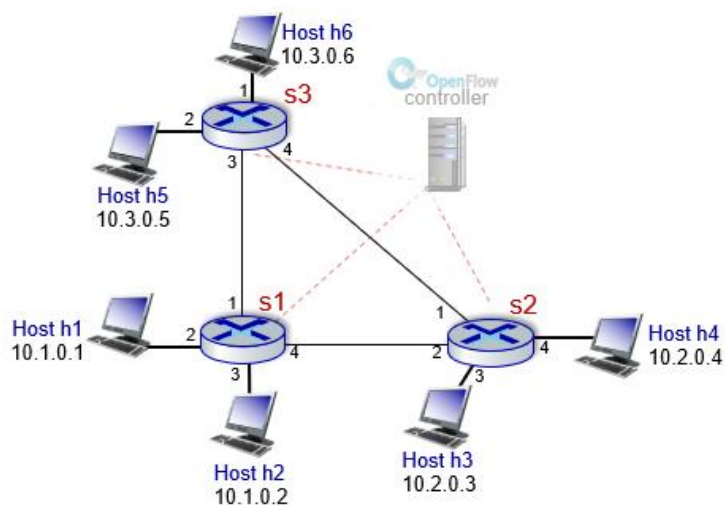
Mobile networks - III



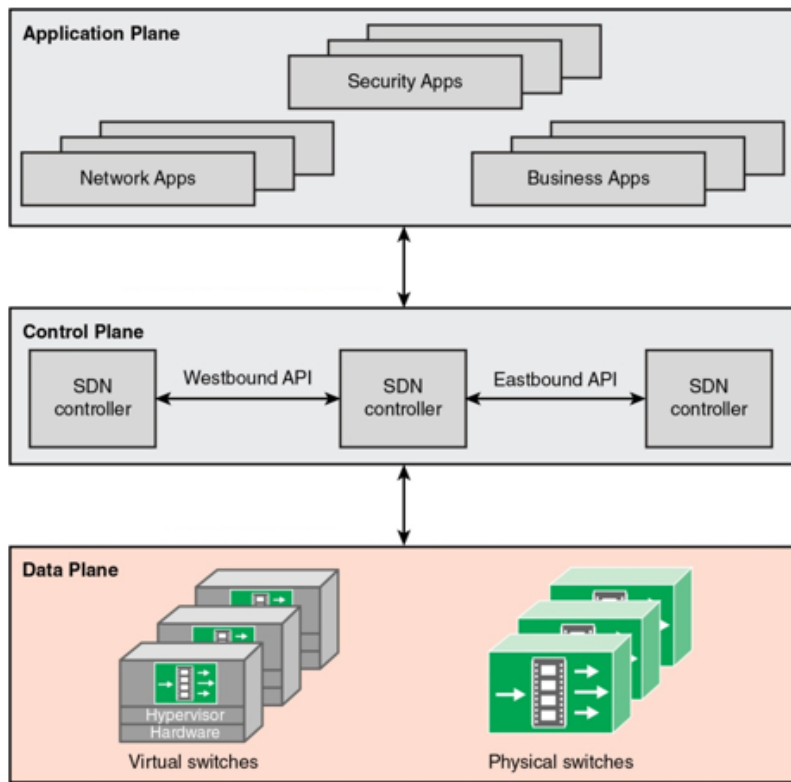
SDN - generalized forwarding



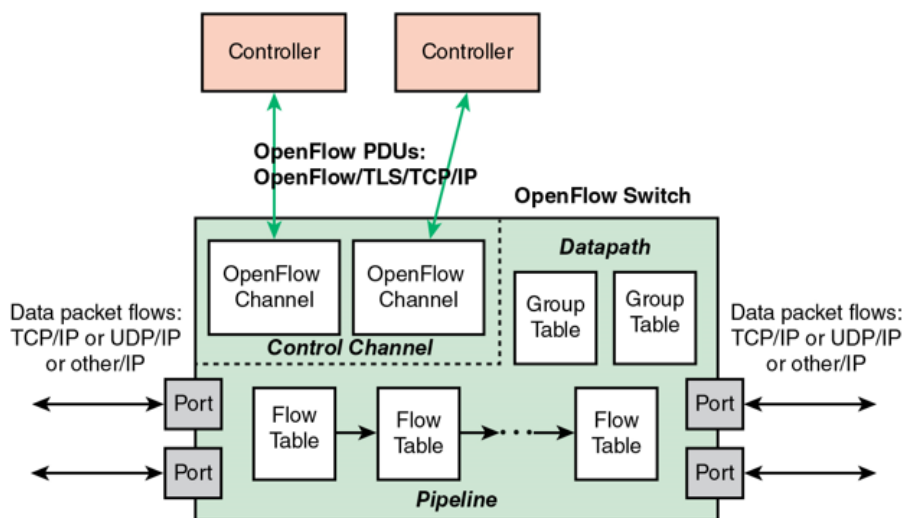
SDN - example



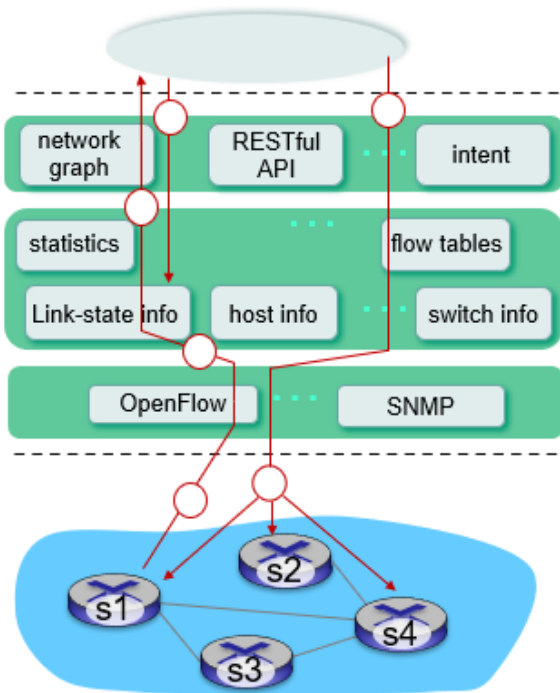
SDN – Architecture



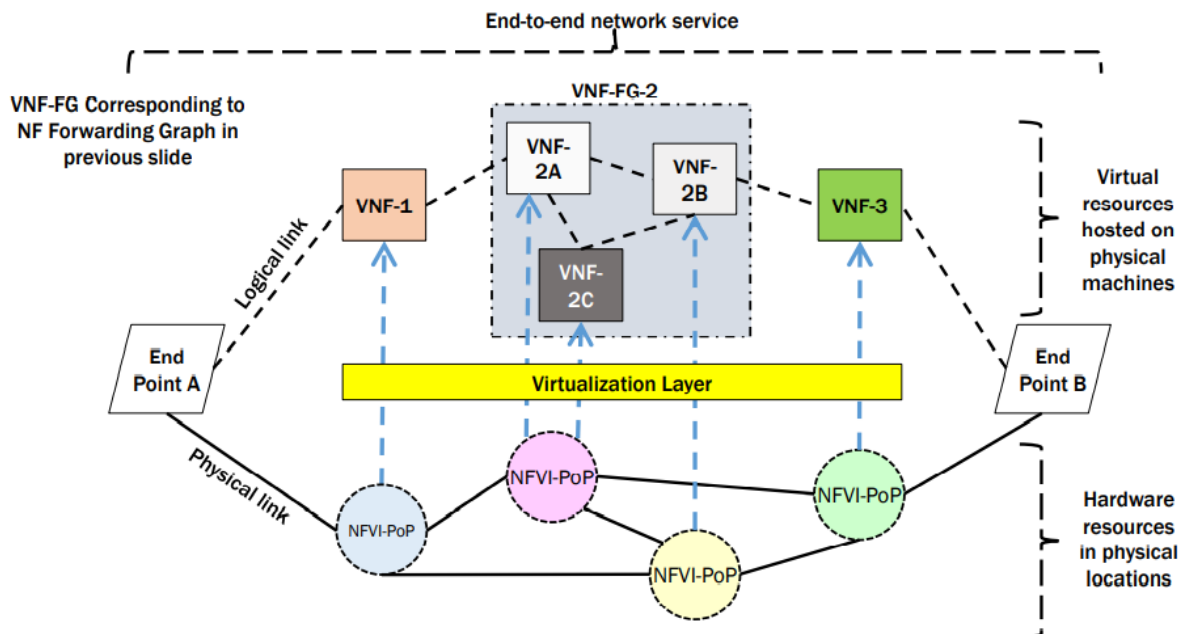
OpenFlow switch



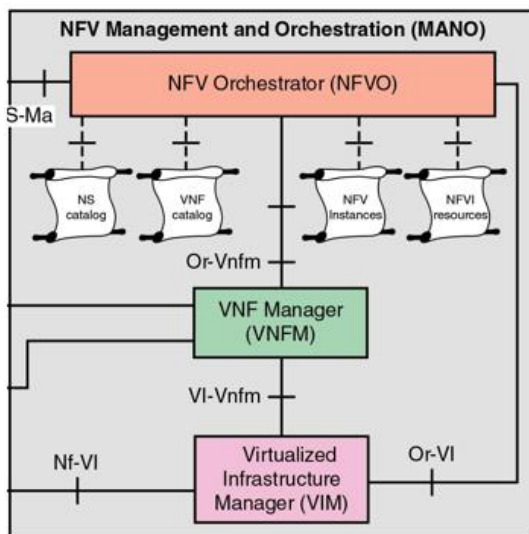
SDN - control/data plane interaction example



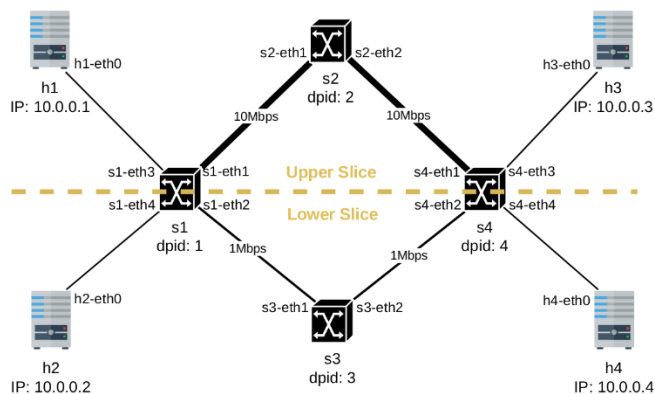
NFV – forwarding graph



NFV – Management and Orchestration



Network slicing



```
def __init__(self, *args, **kwargs):
    super(TrafficSlicing, self).__init__(*args, **kwargs)

    # out port = slice to port[dpid][in port]
    self.slice to port = {
        1: {1: 3, 3: 1, 2: 4, 4: 2},
        4: {1: 3, 3: 1, 2: 4, 4: 2},
        2: {1: 2, 2: 1},
        3: {1: 2, 2: 1},
    }

@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    msg = ev.msg
    datapath = msg.datapath
    in_port = msg.match["in_port"]
    dpid = datapath.id
    out_port = self.slice to port[dpid][in_port]

    actions = [datapath.ofproto_parser.OFPActionOutput(out_port)]
    match = datapath.ofproto_parser.OFPMatch(in_port=in_port)
    # add_flow(self, datapath, priority, match, actions)
    self.add_flow(datapath, 1, match, actions)
    self._send_package(msg, datapath, in_port, actions)
```

```

@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    # install the table-miss flow entry.
    match = parser.OFPMatch()
    actions = [
        parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                ofproto.OFPCML_NO_BUFFER)]
    self.add_flow(datapath, 0, match, actions)

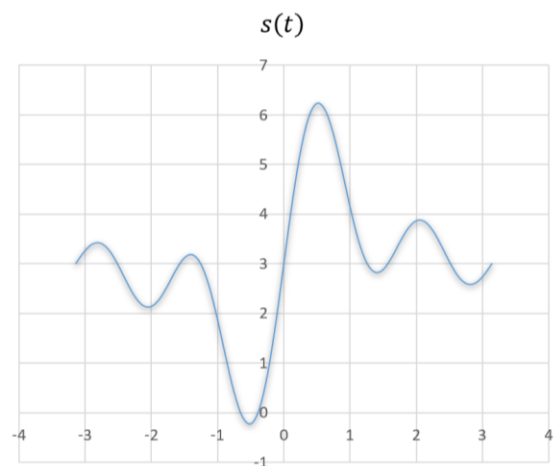
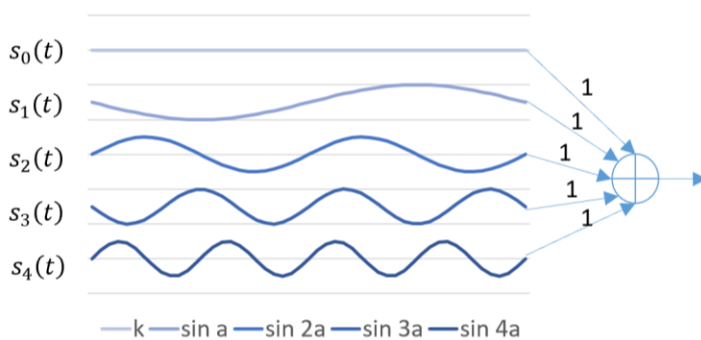
def add_flow(self, datapath, priority, match, actions):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    # construct flow_mod message and send it.
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                          actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                              match=match, instructions=inst)
    datapath.send_msg(mod)

```

Fourier series

$$\frac{1}{2}a_0 + \sum_{n=1}^{\infty} (a_n \cos nt + b_n \sin nt)$$

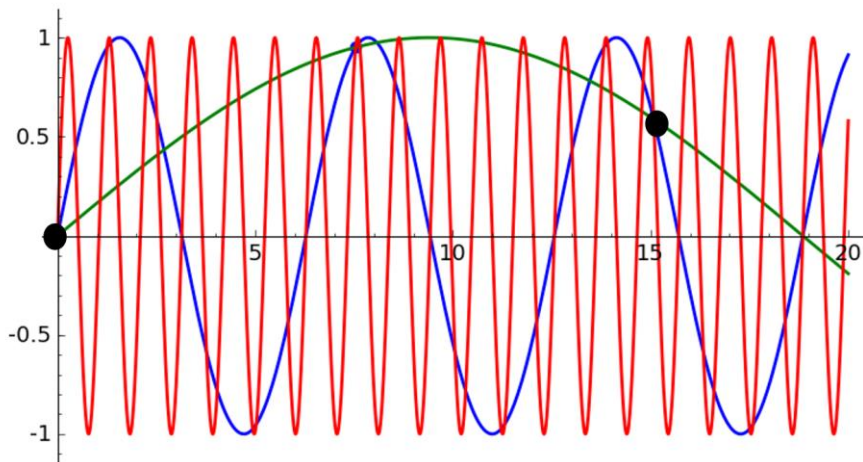


- $s_0(t)$: continuous signal (frequency 0)
 $s_1(t)$: fundamental (frequency $f_0 = 1/T$)
 $s_2(t)$: first harmonic (frequency $f_1 = 2/T = 2f_0$)
 $s_3(t)$: second harmonic (frequency $f_2 = 3/T = 3f_0$)
 $s_4(t)$: third harmonic (frequency $f_3 = 4/T = 4f_0$)

DFT

$$S_f = \sum_{n=0}^{N-1} s_n e^{-j\frac{2\pi f}{N}n} \text{ with } f = 0, 1, \dots, N-1$$

sampling and aliasing



quantization

